
Defending Icagent

Advanced interview preparation for the multi-agent system in this repository

Eight parts · 50 in-depth questions · 30+ diagrams

Every question is about **this system** — the code, the numbers, the decisions, and the bugs that actually happened.

Answers are grounded in the source, not in general theory.

Prepared for **Shikhar Mutta** · September 2026

15 agents · 7 phases · blackboard architecture · zero cost to run

These are the questions a good interviewer asks when this project is on your CV. A bluffer fails most of them.

Contents

PART 1 The System at a Glance

The 60-second answer
Architecture and data flow
The numbers you must know cold
Q1-Q5

PART 2 Architecture Defence

Why a blackboard
The single cycle
The uniform agent contract
Q6-Q13

PART 3 The Two-Tier Split

What `requires_llm` really means
Degradation paths
Score redistribution
Q14-Q20

PART 4 Verification & Trust

Why `g++` is the reliability boundary
Two test cases is not a proof
The retry loop
Q21-Q27

PART 5 Files, State & External Data

Transactional fetch
The tracker's five-block layout
The notes duplicate check
Q28-Q36

PART 6 Cost & Performance

Free-tier arithmetic
The PCH cache
The quadratic read
Q37-Q42

PART 7 Failure & Debugging

Seven real bugs
The empty-response story
Q43-Q47

PART 8 Weaknesses & Scale

What you must volunteer
One user to a thousand
Q48-Q50

The System at a Glance

The material you should be able to produce without thinking — the pitch, the shape, and the numbers.

Interviewers do not test whether you can describe a multi-agent system. They test whether *you built this one*. That means specifics: why this number, what broke, what you rejected. This part is the material you should be able to produce without thinking.

1.1 The 60-second answer

SAY THIS, IN THIS ORDER

“It automates my LeetCode practice workflow — fifteen agents, command line, runs on Windows and Linux, and costs nothing to run.

*It's a **blackboard architecture**: a master agent owns one Context object and passes it down fixed pipelines. No agent talks to another, so ordering is explicit and every run is reproducible.*

The organising decision is a single field, `requires_llm`, that splits the roster into ten agents needing no model and five that do. That's what made zero-cost achievable — with no API key you can still fetch, compile, test, score, archive and commit.

And every model-backed agent is gated by a deterministic verifier: nothing a model writes reaches a file until `g++` and the real test cases agree. That's why a free 20B model is sufficient.”

Four beats: what it does, the architecture, the key decision, why it's cheap. Then stop and let them pick a thread.

1.2 Architecture and data flow

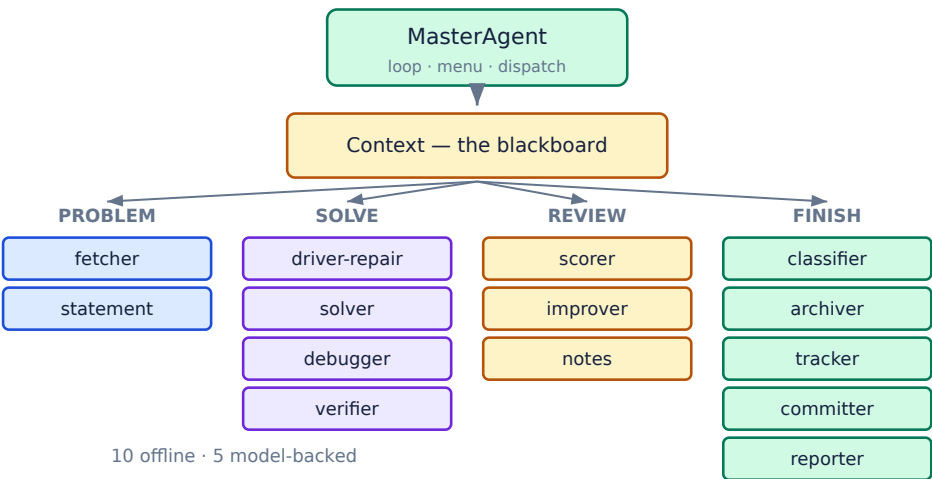


Fig 1.1 — the whole system. Groups are menu sections, not runtime hierarchy: every agent is dispatched by the master and communicates only through the Context.

The `Context` is the only shared mutable state. Each agent reads what it needs and writes one field; nothing else in the system carries information between steps.

```
@dataclass
class Context:
    pid: str # the problem id
    paths: ProblemPaths # every filename, derived – never passed around
    problem: dict | None = None # ← fetcher / statement → scorer, solver, classifier
    report: TestReport | None = None # ← verifier → scorer, debugger, archiver
    score: dict | None = None # ← scorer → improver
    topic: str | None = None # ← classifier → tracker
    history: list[str] = ... # ← every agent appends → `status`
    started: float = ...
```

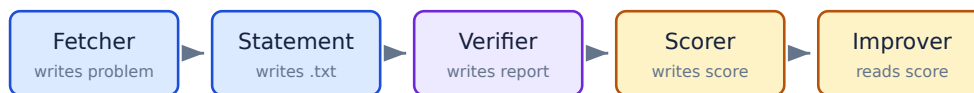


Fig 1.2 — one pipeline, five agents, each enriching the same object. No agent knows which agent ran before it.

1.3 The numbers you must know cold

Quantity	Value	Why it matters in an interview
Agents	15 — 10 offline, 5 model-backed	the zero-cost claim rests on the split
Cycles in the control flow	1 — Verifier $\nrightarrow$ Debugger, capped at 3	cost has a stateable upper bound
Free-tier limit	8,000 tokens/min, 1,000 req/day	the constraint that shaped design
Tokens per score	~1,322 → ~6 scores/minute	why notes isn't automatic
Compile time	5.0 s → 2.8 s with a PCH cache	a real optimisation you measured
Tracker full scan	>120 s → 190 ms	an algorithmic bug you found and fixed
Scorer discrimination	optimal 94/A vs brute force 74/C	evidence the rubric works
Rubric weights	correctness 50, complexity 25, efficiency 15, readability 10	you chose these; defend them
Retry cap	3 attempts, failure fed back each time	bounded, not blind
Problems solved / tracked	935 , 12 starred, 95 topics	the data it operates on
Premium problems	186 of ~3,400 return empty content	a limit you did not work around

THE NUMBER THAT WILL CATCH YOU OUT

If you say “936 solved”, you are quoting the number *before* you found the bug. Row 1 of the tracker is labelled 0000 and carries marks, but LeetCode numbering starts at 1 — so the scan was counting a problem that does not exist. **935** is the corrected figure, and the story of finding it is a better answer than the number itself.

1.4 Questions

Q1 Describe this project in one minute.

Four beats, then stop: **what it does, the architecture, the key decision, why it's cheap.** See §1.1 for the exact wording.

What makes it land is the third beat. “Fifteen agents” is a fact anyone can state; *“the organising decision is one declared field that splits the roster into agents that need a model and agents that don't, and the zero-cost requirement falls out of it”* is a design argument. Interviewers follow design arguments.

Do not enumerate the agents. If they want the list they will ask, and reciting fifteen names burns your minute on the least interesting content you have.

Q2 What was the hardest requirement, and how did it change the design?

Zero cost, added after the architecture was already sketched. It is harder than it sounds because the obvious design — agents that reason about each step — spends a model call per decision, and free tiers are metered per minute.

It changed three things concretely:

- **Agents stopped talking to each other.** A blackboard read is free; an agent-to-agent exchange is two model calls. At 8,000 tokens/minute that is the difference between working and stalling.
- **Capability became a declared property.** Every agent was examined for whether it genuinely needed language understanding. Ten did not — fetching, compiling, running tests, the measured half of scoring, archiving, the tracker, git. That audit is what makes the system usable with no key at all.
- **Verification had to be deterministic.** If correctness depended on the model being right first time, this would have needed a frontier model and a budget. Because g++ is the judge, a free 20B model is adequate.

The general lesson I'd offer: a hard cost constraint is a *design* constraint, not a procurement one.

Q3 How long did it take, and how did you sequence the work?

Seven phases, each landing a working subset rather than a layer. That sequencing is the point — at the end of every phase the system ran.

Phase 0 was the portable execution core: find a compiler, build, run, diff per test case. No agents at all. Doing this first meant every later agent had a reliable primitive to build on, and it forced the Windows/Linux questions to be answered before they could contaminate agent code.

Then: statement rendering (1), fetcher + verifier + the master loop (2) — at which point it was already useful — providers and the scorer (3), the improver (3.5), the three code-generation agents (4), archiving and the tracker (5), notes and reporting (6), then the setup script.

What I would call out: **the model-backed agents came fifth**, not first. The deterministic infrastructure had to be trustworthy before anything unreliable was allowed to depend on it. Building the exciting part first is the standard way these projects fail.

Q4 What does the Context actually contain, and who writes each field?

Eight fields, and **each has exactly one writer** — that single-writer discipline is what makes a mutable shared object safe here.

- `pid`, `paths` — set at construction, immutable thereafter. `paths` is derived, not passed: every filename comes from one place, so no agent invents a path.
- `problem` — written by the fetcher, read by statement, scorer, solver, classifier and notes.
- `report` — written by the verifier, read by scorer, debugger, archiver and tracker.
- `score` — written by the scorer, read by the improver.
- `topic` — written by the classifier, read by the tracker.
- `history` — appended by every agent; the audit trail `status` prints.

The design property worth naming: the dependency structure is visible in that list. You can read off which agents must run before which, and it matches the pipelines exactly. If a field had two writers I would have a race and no way to see it.

Q5 Why is `paths` a derived object rather than strings passed between agents?

Because filenames are a shared convention, and a convention duplicated across fifteen agents is a convention that will drift.

`ProblemPaths` takes the problem id and derives every path from it — solution, driver, input, expected, the statement file at the repo root, the archive directory, the reference solution. One definition, one place to change, and no agent can disagree with another about where a file lives.

It also became the natural home for one genuinely subtle decision. `compile_target()` has to pick what to hand g++, and the original `run.sh` took the first sorted match of `<id>_*.cpp`. That breaks the moment another file shares the shape: `4_better.cpp` sorts before `4_driver.cpp` and has no `main()`, so the build fails with a link error that points nowhere near the cause. The Python version names the driver explicitly and only falls back to the glob.

That is the general argument for the class: it is where knowledge about the file layout lives, so a bug in that knowledge is fixed once.

Architecture Defence

What you rejected, and what rejecting it cost you.

Every architecture question is really the same question: *what did you reject, and what did rejecting it cost you?* An answer that only praises the choice you made is an answer from someone who never considered another.

2.1 Why a blackboard

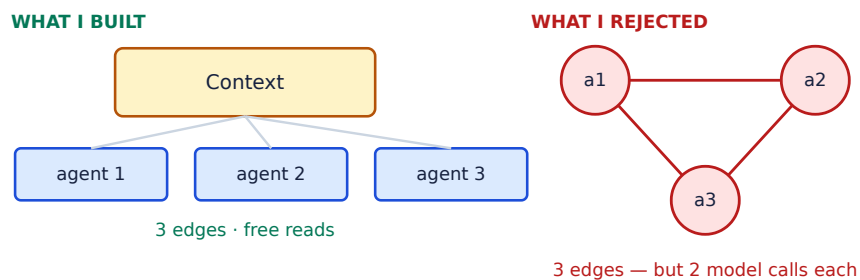


Fig 2.1 — at three agents the topologies look comparable. The difference is that every edge on the right is metered.



Fig 2.2 — the cost of coordination. At 15 agents, a fully connected chat topology would cost 210 model calls per run before any *work* is done. At ~1,300 tokens each that is 273,000 tokens — **34 minutes of the free-tier budget** spent on agents talking.

2.2 The single cycle

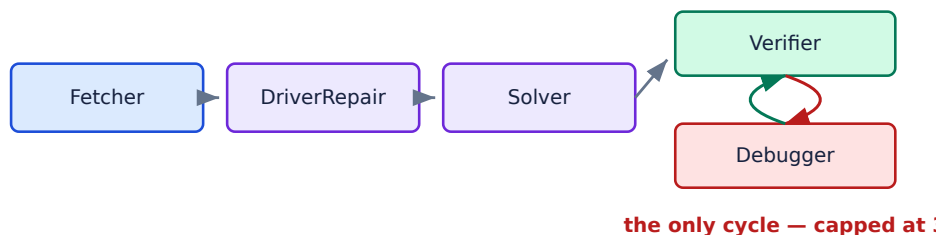


Fig 2.3 — the `solve` pipeline. Everything left of the Verifier is a straight line; the one loop is bounded, so the worst-case cost of the command can be stated before it runs.

2.3 Questions

Q6 You chose a blackboard. Name what would have broken with agent-to-agent messaging, and quantify it.

The free-tier budget would have broken, and it is arithmetic rather than opinion.

Every agent-to-agent exchange costs a model call on each side. With 15 agents, a topology where each coordinates with each is up to 210 exchanges per run. At roughly 1,300 tokens per call — my measured figure for a scoring call — that is about 273,000 tokens spent on *coordination alone*. Against a limit of 8,000 tokens per minute, that is **34 minutes of budget before any actual work**. The system would not have run once.

A blackboard read costs nothing. The coordination cost drops to zero and the whole budget goes to the five agents that genuinely need a model.

What I gave up: no natural parallelism, and the Context schema is a coupling point. Neither bites here — the workflow is inherently sequential (you cannot score before you compile) and there is one developer. If this were a multi-tenant service, the parallelism argument would start to matter.

Q7 Your Context is mutable and passed by reference. What stops agent five corrupting agent two's work?

Nothing enforces it at the type level — and I would say that plainly rather than pretend otherwise. What protects it is a **single-writer discipline**: each field has exactly one agent that writes it, and that mapping is documented and matches the pipeline order.

Three things make the risk acceptable. It is **single-threaded**, so there are no races, only ordering mistakes. The pipelines are **fixed in code**, so the write order is reviewable rather than emergent. And `history` records every agent's action, so a field that changed unexpectedly leaves a trace.

What I would do if this grew: make Context **frozen** and have each agent return a new one with its field replaced — the functional approach makes accidental mutation impossible and gives you every intermediate state for free, which would also improve debugging. I did not do it because with one writer per field and a single thread it buys safety I was not actually lacking, at the cost of allocation churn on a hot path.

Q8 The Context schema is a coupling point. Which field is closest to being a problem?

`score`, because it is the only field typed as a bare `dict` rather than a structured object. Everything else is either a primitive or a proper dataclass — `report` is a `TestReport`, `paths` is a `ProblemPaths`.

That matters because the improver reads `score["better"]` to decide what to write. A dict key is an undeclared contract: rename it in the scorer and the improver breaks at runtime, not at import, and only on the path where a better solution exists — so the tests that matter would pass.

The fix is to promote it to a `ScoreCard` dataclass, which the scorer already has internally; it just serialises to a dict on the way into the Context. That is a half-hour change and the reason it has not happened is that there is exactly one consumer. With two consumers I would do it immediately.

The general point: **the risky coupling is the untyped one**, not the shared object itself.

Q9 You have exactly one cycle. Was acyclicity a goal or an outcome?

A goal, and I would defend it on two grounds.

Cost predictability. An acyclic pipeline has a worst-case cost you can state before running it. With one bounded cycle, `solve` is at most: one fetch, one driver-repair (≤ 3 attempts), one solver (≤ 3), then ≤ 3 debugger iterations — a number I can write down. A system with unbounded cycles has a worst case of “until the budget runs out”, which on a metered free tier is not acceptable.

Debuggability. With a straight line, “what happened?” is answered by reading the pipeline definition. With cycles, it requires a trace, and traces of non-deterministic systems may not reproduce.

The one cycle that survives is `Verifier  $\rightleftharpoons$  Debugger`, and it earns its place because **one side of it is deterministic**. The loop terminates on a fact — the tests pass — not on a model's opinion that it is finished. That is the property that makes a cycle safe, and it is why I would not have allowed a `Solver  $\rightleftharpoons$  Reviewer` loop where both sides are models.

Q10 A reviewer says ``ok`` and ``message`` in `AgentResult` are redundant. Answer them.

They serve different consumers and must never be confused. `ok` is the machine-readable verdict that **control flow** depends on; `message` is a human sentence for the terminal.

Collapsing them means the caller decides what to do next by inspecting text — substring checks, or worse another model call. That is fragile in the ordinary case and actively dangerous here, because a message is the thing most likely to be reworted.

The separation also lets the message be genuinely good. `archiver` can say “completed a partial archive with 5 files” while `ok` is simply true; the master prints one and branches on the other.

I would then concede the real design flaw in that class, because it is more interesting than defending the obvious: `ok()` takes `message` as a positional argument and everything else as `**data`, so an agent passing `message=` inside its data collides with the parameter. The committer did exactly that and raised a `TypeError` on a path that only fired during dry runs. I renamed the field to `commit_message` and audited every other call site — but the signature is still a trap, and a `data: dict` parameter would have prevented it by construction.

Q11 Why does the master implement the same Agent contract as the things it dispatches?

Mostly for uniformity, and I would be honest that the practical payoff is smaller than it sounds.

What it genuinely buys: the roster listing is one loop over a list that includes the master, with no special case; and `run()` gives a single entry point so `python -m lcagent` and an embedded caller take the same path.

What it does not buy: the master is never dispatched *by* anything, so the polymorphism is unused in the direction that would matter. If I were being strict, the master is an orchestrator that happens to satisfy an interface, not an agent.

I kept it because the cost is three lines and it keeps one invariant true — *everything in the roster is an Agent* — which makes the ``a`` command and any future introspection trivially correct. I would not defend it as a deep design decision, and claiming it was one is the kind of overreach interviewers notice.

Q12 Why not use LangChain, AutoGen, or CrewAI?

Three reasons specific to this project, in order of weight.

The dependency budget. A hard requirement was running on Windows and Linux with minimal setup. The system has exactly *one* third-party dependency, `openpyxl`, and only for the spreadsheet — everything else is the standard library. A framework brings a dependency tree that has to install correctly on a machine I cannot test, which directly undermines the portability requirement.

The abstractions do not fit. These frameworks are built around agents that converse and around dynamic planning. My design decision was the opposite on both counts: fixed pipelines and no inter-agent messaging, for the cost reasons in Q6. I would have been fighting the framework's grain to get less than I wanted.

Learning. This is a project to understand multi-agent design. Delegating the agent loop to a library would have removed the part I was trying to learn.

What I would use them for: rapid prototyping, or a system that genuinely needs dynamic planning and tool routing. This one needs neither, and the total orchestration code is a few hundred lines.

Q13 Adding an agent should be cheap. Prove it.

Three steps, and nothing existing changes:

```
# 1. one new file – implement the contract
class MyAgent(Agent):
    name = "mine"
    role = "One line – this appears in the roster"
    requires_llm = False
    def run(self, ctx: Context, **kwargs) -> AgentResult:
        return self.ok("did the thing", detail=42, artifacts=[])

# 2. master.py – import, construct in __init__, add a cmd_mine method
# 3. add one MENU row and one _ALIASES entry
```

That is the blackboard property from Q6 paying off: the new agent couples to the *Context schema*, not to any other agent, so no existing agent needs to learn it exists. In a message-passing design this would be $O(n)$ — every peer that should collaborate with it needs updating.

The honest caveat: step 2 touches `master.py` three times, so the master is a mild bottleneck for change. A registry — agents declaring themselves via a decorator — would make it truly $O(1)$. With fifteen agents that is over-engineering; at fifty it would not be.

Evidence it actually is cheap: phase 6 added two agents, `notes` and `reporter`, and modified no existing agent's logic.

The Two-Tier Split

The decision the project turns on — including the subtlety a sharp interviewer will find.

The two-tier split is the decision the whole project turns on, and it has a deliberate subtlety that a sharp interviewer will find: two of the ten “offline” agents call a model.

3.1 What `requires_llm` really means

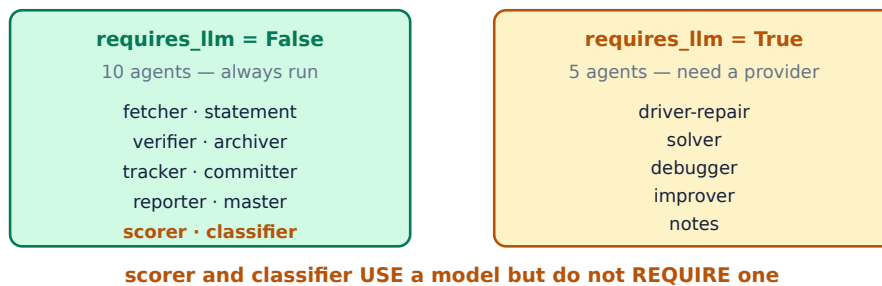


Fig 3.1 — the field means *required*, not *used*. The two amber agents on the left are the interesting cases.

```
# classifier.py – three levels, not two
def run(self, ctx, **_):
    known = tracker_io.vocabulary(minimum=2)      # 36 topics already in the sheet
    provider = get_provider()
    if provider.available()[0]:
        try:
            topic = self._ask(ctx, known, tags, provider)    # BEST: model picks
            if topic:
                return self.ok(topic, topic=topic, source="model")
        except ProviderError:
            pass
            # fall through – do NOT fail
    topic = self._from_tags(tags, known)          # GOOD: deterministic tag mapping
    if not topic:
        return self.fail("could not determine a topic (no tags, no model)") # FAIL
    return self.ok(topic, topic=topic, source="tags")
```

3.2 Score redistribution

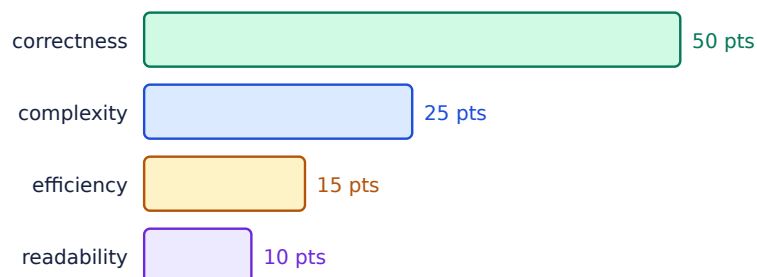


Fig 3.2 — the rubric. Correctness and efficiency are **measured**; complexity and readability need a model. With no provider the measured 65 points are scaled back up to 100.

3.3 Questions

Q14 The scorer calls a model but is marked `requires_llm = False`. Justify that.

Because the field means **required**, not **used**. It answers one question: *can this agent do its job with no provider configured?* The scorer can.

Its rubric has four components. **Correctness** (50 points) comes from the verifier's test results and **efficiency** (15) from a benchmark against the archived reference solution — both measured, no model involved. **Complexity** (25) and **readability** (10) need a model to read the code. With no provider, the scorer computes the 65 measured points, marks the card `degraded`, and scales those components back up to 100 so the grade is still on a familiar scale.

So without a key you still get a real correctness-and-speed grade; you lose the complexity verdict and the better-approach suggestion.

The counter-argument I would concede: this makes the flag's meaning subtle, and someone reading the roster sees “offline” next to an agent that makes network calls when it can. A cleaner design would be a three-valued `capability = REQUIRED | ENRICHING | NONE`. The boolean was right at the time and would be the wrong choice at twice the size.

Q15 What is the failure mode of a system where 'requires' means 'prefers'?

Silent quality variation. The same command produces a materially different result depending on whether a key happened to be set, and if the output does not say which path it took, nobody can tell.

Concretely: `finish` on a machine with a key classifies the topic using the sheet's own 36-term vocabulary; without one it maps the raw LeetCode tag. Both write to the tracker. Six months later the topic column has two populations in it and no record of why.

That is exactly why every degradable result **carries its provenance** — the classifier returns `source="model"` or `source="tags"`, and the score card carries `degraded: bool`. A degraded answer that is labelled is useful; an unlabelled one is a data-quality bug that surfaces much later.

What I did not do, and would: persist that provenance *into the tracker*, not just the session. Right now the sheet records the topic but not how it was derived, so the two populations are still indistinguishable after the fact.

Q16 Defend reporting 94/A from a degraded card instead of 65/100 with a caveat.

The argument for redistribution is **comparability**. A grade's job is to answer “is this solution good?”, and a scale that silently changes length depending on configuration makes scores incomparable across runs. If one machine reports 65 and another 94 for identical code, the number is worthless for tracking progress.

So the measured components are scaled to fill 100 and the card is flagged `degraded`, which the renderer displays.

Now the honest weakness, because this is the better half of the answer: **the redistributed score is not measuring the same thing**. A full card weights complexity at 25 — the component that catches an $O(n^2)$ brute force. Remove it and redistribute, and a brute force that passes the tests quickly scores *higher* on a degraded card than on a full one. The degradation is not neutral; it systematically favours exactly the solutions the scorer exists to catch.

What I would change: keep the redistribution for comparability but **cap a degraded card below A**, since the evidence for an A is precisely what is missing.

Q17 Which model-tier agent would you promote to offline, and what would you lose?

`driver-repair`, and it is the only one that is close.

Its job is filling `// TODO: parse ...` scaffolding in a generated driver — reading an input format and constructing the right C++ type. A large share of those TODOs are a small set of shapes: `vector<int>`, `vector<string>`, `vector<vector<int>>`, a bare `int`. A template library keyed on the declared parameter type would handle most of them deterministically, and the model would become the fallback for the unusual cases — trees, linked lists, custom classes, special judges.

What I would lose: the long tail, which is where the interesting failures live. Tree problems need a level-order deserialiser; problems with multiple valid outputs need a special judge rather than string equality, and those are not TODO-marked at all, so they cannot be pattern-matched.

The others are not candidates. `solver` and `debugger` write arbitrary algorithms; `improver` writes a specific named approach; `notes` judges whether a technique is worth remembering. All four are irreducibly language tasks.

Q18 You claim 10 of 15 agents are offline, but the fetcher needs the network. Is that honest?

The claim is precise but easy to misread, so I would restate it: the flag means **no model provider required**, not **no network required**. Those are different resources with different costs — a LeetCode GraphQL call is free and unmetered; a model call is neither.

The fetcher is genuinely network-dependent, but it degrades in its own dimension. It tries the on-disk cache first, then the local archive of ~3,350 solutions, and only then the network; `--offline` forces it to skip the network entirely. So for a previously-fetched or archived problem it works with no connection at all.

Where I would accept the criticism: the roster prints a two-column split labelled `offline / model`, and `offline` naturally reads as “works with no connectivity”. A third column for network dependence would be more honest. The labels themselves came from earlier feedback — they were originally `free / LLM`, which was worse, because it implied the model tier costs money when on the configured provider it does not.

Q19 How would you test the degradation paths?

By making the dependency injectable and forcing each path deliberately — which is exactly what I did for the one dependency that has a hard failure.

For `openpyxl`, I stubbed `builtins.__import__` to raise `ModuleNotFoundError` for that module only, then ran the three agents that touch the tracker. That found a real gap: the reporter degraded with a cryptic message and the tracker would have raised a bare traceback. It now produces an actionable error, and the classifier falls through to tag mapping and still returns a topic.

For the provider path, the same idea with a stub `Provider` whose `available()` returns false, or whose `complete()` raises `ProviderError`. I used a fake provider to exercise nine branches of the notes agent — empty reply, fenced output, duplicate refused, force override, and so on — without spending a single real call.

What is missing and I would add: these are ad-hoc scripts rather than a suite. The right version is a fixture that parametrises over `{no provider, provider errors, no openpyxl, no network, no git}` and asserts each agent's declared behaviour, run in CI. Degradation is a feature, and untested features rot.

Q20 Someone adds an agent and forgets to set `requires_llm`. What happens?

It defaults to `False` on the base class, so the agent claims to work offline. If it then calls `get_provider().complete()` with no provider configured, it raises — and the master's degradation logic will not have protected it, because it was never told to.

That is a real design weakness and I would name it as such: **the safe default is the wrong default**. `False` is the permissive value, so the failure mode of forgetting is a runtime error on a user's machine rather than a caught condition.

Three fixes, in increasing strength. Make it **abstract**, so a subclass must declare it — a two-line change and the one I would actually make. Or **derive** it by checking at construction whether the agent's module imports the provider. Or **assert** it in the roster: a test that instantiates every agent, calls `run()` with no provider, and requires that `requires_llm = False` agents do not raise `ProviderError`. The third catches the mistake even if someone declares it wrongly, which the first two do not.

Verification & Trust

The one-sentence reliability argument — and the specific place it fails.

The reliability argument for this whole system is one sentence: *nothing a model writes reaches a file until g++ and the real test cases agree*. Expect to be asked where that argument fails — and it does fail, in a specific and interesting place.

4.1 Why g++ is the reliability boundary

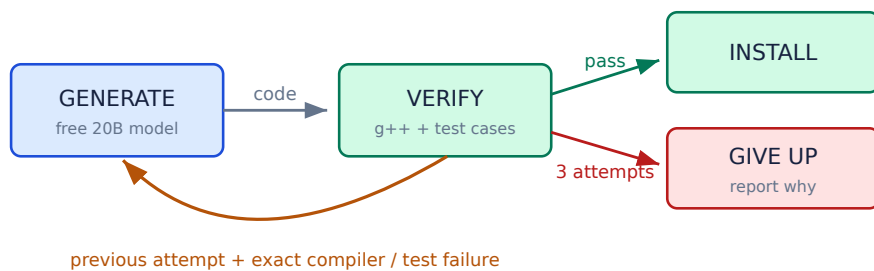


Fig 4.1 — the loop shared by driver-repair, solver and debugger. `MAX_ATTEMPTS = 3`, and the retry carries the failure rather than resampling blindly.

THE ARGUMENT, AND ITS LIMIT

Argument: the system's correctness is bounded by the *verifier's* correctness, not the model's. g++ and a test runner cannot hallucinate, so an unreliable generator becomes a reliable component.

Limit: the verifier only checks what it can see, and what it can see is **two example test cases** scraped from the problem statement. A solution that passes both can still be wrong on an edge case, wrong on a large input, or accidentally correct. “Verified” here means *passes the examples and compiles* — nothing stronger.

4.2 Questions

Q21 Your reliability rests on 'g++ and the tests agree'. Where does that argument fail?

In the strength of the test set. LeetCode statements typically supply **two example cases**, and those are what the verifier checks. So the guarantee is “compiles and passes two examples”, which is much weaker than “correct”.

Three concrete gaps:

- **Edge cases** — empty input, single element, maximum constraint values, duplicates, negative numbers. Examples rarely cover these.
- **Performance** — the examples are tiny, so an $O(2^n)$ solution passes them instantly and would time out on the real judge. My efficiency component cannot see this either, for the same reason.
- **Accidental correctness** — a solution that returns a constant that happens to match both examples.

What I actually do about it: the improver's result is labelled as verified against the example cases specifically, not as correct. And when I wanted real confidence in one solution, I stress-tested it separately — 214,348 random cases against a brute-force oracle, zero mismatches. **That is the missing feature:** a generator producing random and worst-case inputs, checked against a known-good implementation. It would strengthen the verifier, fix the efficiency component, and make “verified” mean something.

Q22 MAX_ATTEMPTS = 3. Derive that number or admit it was arbitrary.

It was chosen, not derived, and I would say so — then give the reasoning that made it defensible.

The trade-off is between recovery and waste. Attempt 2 is genuinely valuable because it is solving a different, easier problem: the prompt now contains the exact compiler error and the failed code, so it is “fix this” rather than “write this”. Attempt 3 catches a second independent mistake. Beyond that, observed failures were *not* converging — the model was cycling between two wrong approaches, and more attempts meant more tokens for the same outcome.

There is also a hard constraint: each attempt is a full call, so 3 attempts $\times$ $\sim 1,300$ tokens is already a meaningful fraction of an 8,000 token/minute budget. A cap of 10 would make a single failing command consume the whole minute.

What would make it principled: instrument it. Log attempt-number versus success across a few hundred runs and set the cap where the marginal success rate stops justifying the tokens. I have not done that, and without the data **3 is a judgement call with a rationale, not a measurement.**

Q23 Your retry fixes case 3 and breaks case 5. Does the loop notice?

No — and that is a genuine bug in the design, not just a missing feature.

The loop's condition is binary: `report.ok`. An attempt that goes from “cases 3 and 5 fail” to “case 5 fails” and one that goes to “case 7 fails” are indistinguishable to it. Both are “still failing”, so it retries with the new failure fed back — and can oscillate between two broken versions until the cap ends it.

The fix is the **progress invariant**: track the *set* of failing cases, not the boolean. Then you can assert that each attempt does not increase the failure count, and keep the best attempt so far rather than the most recent one. If attempt 3 is worse than attempt 2, install attempt 2.

The data is already available — `TestReport` holds per-case results, and the debugger already feeds the specific failing cases into the prompt. It is the *loop* that ignores them, not the reporting. That is maybe twenty lines, and it is the single highest-value change I could make to that loop.

Q24 NOISE_FLOOR_MS = 8.0 with BENCH_REPEATS = 5. Defend both numbers.

The noise floor is the honest part. Example test cases are tiny, so both the candidate and the reference solution finish in a few milliseconds — well inside process startup, scheduler jitter and cache effects. Below 8 ms the measurement carries no signal, so rather than report a meaningless ratio the scorer says so explicitly: “4.1 ms vs 3.6 ms — both below the 8 ms noise floor, example tests are too small to differentiate”. Refusing to report a number you do not believe is the right behaviour.

Five repeats is weak and I would concede it. Five samples of a noisy quantity gives a poor estimate, and I take a summary statistic across them rather than a distribution. The defensible version takes the **minimum** rather than the mean — minimum is the standard choice for timing because noise is one-sided, it can only make a run slower.

The deeper issue is that *tuning these constants is treating the symptom*. The real problem is that the inputs are too small to measure, and no amount of repetition fixes that. With a stress-test generator producing large inputs, the noise floor becomes irrelevant and the measurement becomes meaningful.

Q25 Your efficiency component gave a brute force 15/15. Why did you ship it?

Because the *grade* was still right, and I could say precisely why.

Tested on problem 4 — Median of Two Sorted Arrays — an $O((m+n) \log(m+n))$ merge-and-sort scored **74/C** while the optimal binary-search partition scored **94/A**. The discrimination came almost entirely from **complexity: 5/25 versus 25/25**, which reads the code rather than timing it. Efficiency gave both full marks because both run the two tiny examples in about 4 ms.

So the component is correct-by-accident on one axis and the rubric compensates. I shipped it because removing it would have lost the case it *does* catch — a genuinely slow implementation of the right algorithm — and because the alternative was blocking the phase on a stress-test generator I had not built.

What I would say next, unprompted, because it is the more interesting half: **shipping a component that is right for the wrong reason is a decision with a shelf life**. It is acceptable while documented and while the rubric compensates. It stops being acceptable the moment someone trusts the efficiency number on its own — and nothing in the system stops them. That is why it is written down as a known gap rather than left as a quiet compromise.

Q26 How does the debugger know which cases failed, and what does it send the model?

The verifier produces a `TestReport` with per-case results, and the debugger sends the **specific failing cases** — input, expected output, actual output — not a summary.

Recovering the per-case input is less trivial than it sounds. The driver reads all cases from one input file, so the failure has to be attributed back to the right slice. The driver echoes each case to stderr separated by `---`, and `split_case_inputs()` reconstructs the mapping. Without that you can only tell the model “case 3 failed” without being able to say what case 3 *was*, which is close to useless.

Why it matters: the difference between “*your solution fails 2 of 8 cases*” and “*on input [1,3] [2] you returned 2, expected 2.00000*” is the difference between a resample and a repair. The second is a diagnosable statement, and in that specific example it points straight at a formatting bug rather than an algorithmic one — which is a real bug I hit and fixed with `setprecision(5)`.

Q27 The improver writes to your solution file. What if it produces something worse?

It cannot install something worse in the sense that matters, because **installation is gated on verification**. The order is deliberate:

1. Write the candidate to `data/improved/<id>_better.cpp` — never directly to the working file.
2. Back up the current solution to `data/replaced/<id>_before_improve.cpp`.
3. Compile and run the candidate against the real test cases.
4. Install into `<id>.cpp` **only if it passes**.

So a candidate that fails to compile or fails a case never touches your file, and the previous version is recoverable either way.

Now the honest limit, which is the same one as Q21: “passes” means the two example cases. A candidate could be asymptotically better and subtly wrong on an edge case the examples do not cover, and it would install. The backup is what saves you, not the verification.

This is not hypothetical — during testing my optimal solution for problem 4 was overwritten by a stale editor buffer, and it was recoverable precisely because both `data/improved/` and `data/replaced/` held byte-identical copies. The safety net earned its keep on the first real incident.

Files, State & External Data

Writing into files that took months to build by hand — and the rules that follow from that.

This system writes into files that took months to build by hand: a tracker spreadsheet, three notes files, an archive. Those get stricter rules than anything it generates, and the reasoning behind those rules is very interviewable.

5.1 Transactional fetch

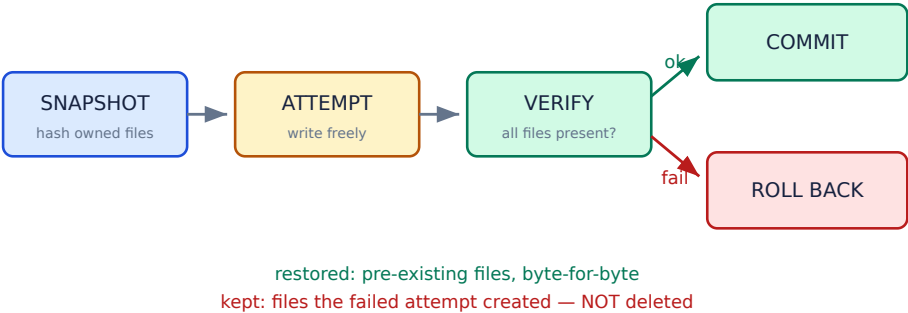


Fig 5.1 — `_rollback()` returns `(restored, kept)`. The second half of that tuple is the design decision.

5.2 The tracker's five-block layout

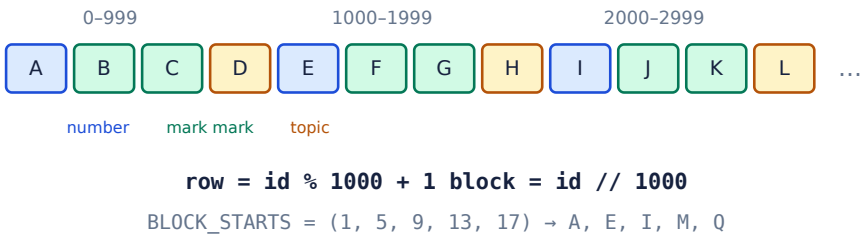


Fig 5.2 — five side-by-side blocks of 1000. Verified against known rows: 2859 → row 860 block I-L, and 4 → row 5 block A-D.

5.3 Questions

Q28 Your rollback deliberately keeps partial files. A reviewer calls that leaving garbage. Defend it.

The guarantee users actually want is “*my existing work is untouched*”, not “*the directory is pristine*”. Those are different, and the second one destroys evidence.

When a fetch fails halfway, the partial output is often the only clue to *why*. Deleting `4046_input.txt` because the expected-output step failed removes the artefact that shows the input parsed fine and the failure was downstream. For a developer tool used by the person debugging it, that is the wrong trade.

So `_rollback()` returns `(restored, kept)` and the master reports both explicitly:

```
restored unchanged: 2859.cpp, 2859_driver.cpp
partial files kept for inspection: 4046_input.txt
none of your existing work was modified or deleted
```

The reviewer's concern is legitimate about the *next* run, not this one — stale files could confuse a later fetch. That is handled separately: `_replace_previous()` clears the previous problem's files from the root before scaffolding new ones, so the leftovers are cleaned when it matters and preserved when it helps. This behaviour was a direct response to user feedback, twice: first “don't modify old files”, then “also don't delete failed files”.

Q29 What is your definition of 'atomic' here, and where does it not hold?

My definition: **after a failure, every file that existed beforehand is byte-for-byte what it was**. Not “no side effects” — deliberately, per Q28.

Where it does not hold, and I would volunteer these rather than wait:

- **Crash mid-write.** The snapshot is in memory. If the process is killed during the attempt, nothing restores anything. A durable journal would fix it; for a local CLI I judged the complexity not worth it.
- **Files outside the owned set.** `_owned()` enumerates the five files for a problem id. Anything the fetcher touches outside that — the cache directory, for instance — is not snapshotted and not restored.
- **The rescue step.** `_replace_previous()` copies unarchived solutions to `data/replaced/` before clearing them. If *that* copy fails, I am mid-way through a destructive operation with no snapshot of its own.

The last one is the real hole. The fix is to order it correctly — copy first, verify the copy exists and matches, only then clear — which is a small change I have not made. Being able to name the gap in your own transactional guarantee is worth more than claiming you do not have one.

Q30 Walk me through locate(). What breaks at problem 5000?

`locate(pid)` returns `(row, column): row = n % 1000 + 1` and `col = BLOCK_STARTS[n // 1000]`, where `BLOCK_STARTS = (1, 5, 9, 13, 17)` — columns A, E, I, M, Q. The sheet is five side-by-side blocks of a thousand problems, each block being four columns: number, mark, mark, topic.

I did not infer that from documentation; I derived it by inspection and then **verified it against known rows** — 2859 lands at row 860 in block I-L, 4 at row 5 in block A-D, 3151 at row 152 in block M-P. That verification step is the part worth mentioning, because a silent off-by-one here would corrupt the user's records invisibly.

At 5000 it raises, deliberately: `n // 1000 == 5` is out of range for a 5-tuple, and rather than let an `IndexError` escape it raises a `ValueError` saying the problem is beyond the sheet's 5,000 rows. LeetCode is around 3,700 problems, so the ceiling is years away — but failing loudly with an explanation beats writing to column U, which does not exist, or wrapping silently to block 0, which would overwrite a real row.

Q31 You found a 'solved problem 0'. How, and what does it say about your other assumptions?

I found it while building the reporter. The solved count came out at 936, and when I dumped the starred set to resolve titles, the lowest id was **0**. LeetCode numbering starts at 1, so that could not be real.

The cause: row 1 of block A is labelled `'0000'` and carries  and  marks, but has no topic. Under `row = n % 1000 + 1`, row 1 maps to problem 0. The scan was faithfully counting a row that exists in the sheet and not in the world.

What it says about my other assumptions — and this is the better half of the answer — is that **I had verified the mapping but not the data**. I checked that `locate()` put known problems in the right cells, which is a different claim from “every marked cell corresponds to a real problem”. The first is about my arithmetic; the second is about someone else's spreadsheet, maintained by hand over months, which will contain exactly this kind of artefact.

So the count is now **935**, and the general lesson I took: when reading externally-maintained data, validate the *values* against the domain, not just the *addressing* against your formula.

Q32 Your notes duplicate check uses a 0.6 token-overlap threshold. Derive it.

Chosen from the cost asymmetry rather than from data, and the asymmetry is the argument.

These files hold 50, 16 and 15 entries across 935 solved problems — roughly **one note per twenty problems**. A **false negative** (a near-duplicate is appended) permanently pollutes a hand-curated file the user has to read forever. A **false positive** (a genuine entry is refused) costs one `--force` flag, and the refusal message names the entry it thinks is a duplicate, so the user can judge immediately.

Those costs differ by orders of magnitude, so the threshold is set to over-refuse. 0.6 of the *smaller* title's meaningful tokens is aggressive on purpose.

It is one of three independent signals, which matters more than the number: the problem already being cited (LC 258 appearing anywhere), an exact normalised title match, or the overlap. Any one refuses. And the model is separately shown every existing title so it can decline on its own — three layers, because appending to a hand-maintained file is the least reversible thing this system does.

What would make it principled: run it over the 81 existing entries pairwise and check that no two legitimate entries exceed the threshold. I have not, so 0.6 is a reasoned choice, not a measured one.

Q33 Why not just append to the notes file? Why reverse-engineer three different formats?

Because these are the user's files, not the system's, and a tool that reformats months of hand-written notes to suit itself is a tool nobody keeps using.

The three formats differ genuinely, and two of the three are **insertions, not appends** — which is the non-obvious part:

- `imp DS.txt` — entries 51. Title, separator $U+2500 \times 62$, appended at the end.
- `formula.txt` — entries [17] Title, separator $- \times 40$, and the file ends with a $= \times 40$ **footer**. Appending to the end would put the new entry *below* the footer.
- `algo_name.txt` — a plain numbered list under an Algorithms heading, with unrelated loose notes below it. The insertion point is after item 15, not at end of file.

Getting that right meant reading the actual bytes — separator lengths, trailing newlines, whether numbers are padded. Then I tested every append against **copies** and diffed, so the real files were never opened for writing until the format was proven byte-exact.

The prose is also reflowed to the files' own 74-column style, with one rule: a line the model indented is code and passes through untouched. My first version wrapped line-by-line and left orphan fragments, because the model brings its own line breaks.

Q34 Every tracker write takes a backup, even a no-op. Justify the cost.

The cost is a 57 KB file copy, taking milliseconds. The thing it protects is months of hand-maintained records that exist in exactly one place. That asymmetry ends the argument.

The reason it happens even on a no-op is that **“this write will not change anything” is a prediction**, and the paths where it turns out to be wrong are exactly the paths nobody tested. If the backup is conditional, the condition is another thing that can be wrong, and its failure mode is silent data loss. Unconditional is one fewer branch to get right.

I also verified the file was safe to round-trip *before* trusting openpyxl with it at all: no charts, no images, no conditional formatting, no merged cells, no defined names — because openpyxl silently drops several of those on save. Checking that first was more important than the backup, since a backup does not help if you do not notice the corruption for a month.

The related rule: an existing ★ is never overwritten with a ✓. It is the user's own annotation carrying meaning the system does not have, so the system does not get to decide it is obsolete.

Q35 105 archive files contain '*' in the filename. Why did you work around it rather than rename?

Because they are the user's own revisit markers, and renaming them is a decision about *their* convention, not a bug fix. `1009*.cpp` and even `1157**ST.cpp` encode something — probably difficulty or a re-review flag — that I do not have the meaning of. Silently normalising them destroys information to make my code simpler.

So the reporter's lookup handles it: `glob <id>*`, then filter on a digit boundary so asking for `448` cannot match `4480`. Ten lines.

The part I would raise unprompted, because it is the actual problem: *** is illegal in a Windows filename**. Those 105 files cannot be checked out on Windows at all, which directly undermines a stated requirement of this project. I surfaced it, explained the consequence, and left the decision with the user — it is a repository-wide rename that touches their history.

The general position: **work around other people's conventions, escalate when they conflict with a requirement, and do not unilaterally rewrite data you do not understand.**

Q36 The archive doubles as your scorer's reference corpus. What risk does that create?

Poisoning. The scorer benchmarks a candidate against the archived solution for the same problem, so if a bad solution gets archived, every future comparison against that problem is silently wrong — and wrong in the flattering direction, because a slow reference makes anything look fast.

Two mitigations exist. Archiving is **gated on passing tests**, so a broken solution cannot enter through the normal path. And the corpus was a bulk import of ~3,350 solutions in a single commit, so its baseline quality is independent of this system's behaviour.

Now the case where it actually bit, which is the honest answer: during testing, a brute-force solution for problem 4 was archived, replacing the optimal one. The tests passed — it *is* correct — so the gate did not fire. Every future efficiency comparison for problem 4 is now against an $O((m+n) \log(m+n))$ reference instead of the optimal $O(\log(\min(m,n)))$.

What that reveals: **“passes the tests” is the wrong gate for a reference corpus.** A reference should be the best known solution, not merely a correct one. The fix is to refuse to overwrite an archived solution with one that scores lower, or to keep the higher-scoring version — the score card is already computed and already on disk.

Cost & Performance

Defending 'free' with arithmetic, and two performance numbers that came from real bugs.

“Free” is a claim you will be asked to defend with arithmetic. So is every performance number you quote — and two of yours came from finding real bugs.

6.1 Free-tier arithmetic

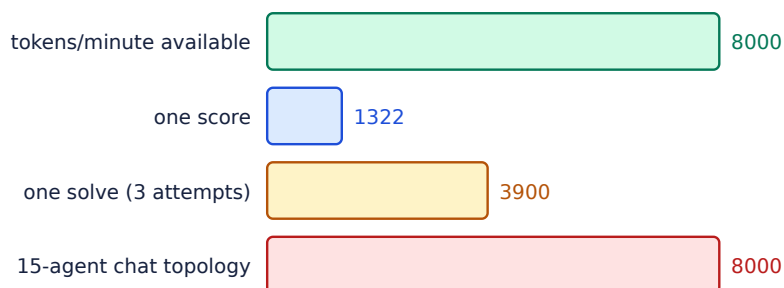


Fig 6.1 — Groq free tier, measured. The red bar is capped at the limit; a fully-connected chat topology would need ~273,000 tokens per run, or 34 minutes of budget, before doing any work.

6.2 The two performance bugs

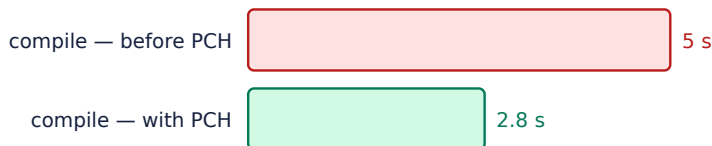


Fig 6.2 — precompiling `<bits/stdc++.h>`. Counter-intuitively, `-O0` was *slower* than `-O2` — the header parse dominates, not codegen.

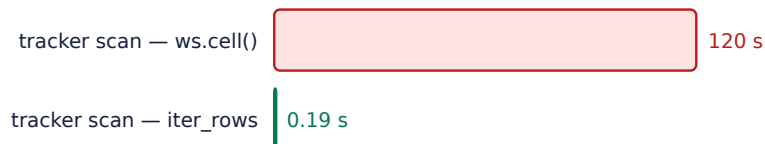


Fig 6.3 — the quadratic read. `read_only=True` worksheets stream rows, so random access rescans from the top every call.

6.3 Questions

Q37 Prove the 'free' claim with arithmetic.

Two independent legs.

Leg one: most work needs no model. Ten of fifteen agents are deterministic. With no key at all you can fetch a problem, render the question file, compile, run the tests, get a correctness-and-speed score, archive, update the tracker, report progress and commit. That is the full core loop.

Leg two: the rest fits in a free tier. Measured on Groq: 8,000 tokens/minute and 1,000 requests/day. A scoring call is ~1,322 tokens, so about **six scores per minute**. A `solve` at three attempts is ~3,900 tokens. A practice session is maybe ten problems — well inside 1,000 requests/day, and the per-minute limit only binds if you run several commands back to back.

The design decisions that keep it inside the budget: no agent-to-agent messaging (Q6), a retry cap of 3, and `notes` not running automatically on every `finish` — the answer there is usually “nothing new”, so an automatic call would spend ~1,300 tokens per finish to write nothing.

And the fallback: Ollama runs locally with no account and no network, so “free” survives even the provider disappearing.

Q38 -O0 compiled slower than -O2. Explain.

Because with `<bits/stdc++.h>` the bottleneck is not code generation — it is **parsing and instantiating the entire C++ standard library**. That header pulls in essentially all of `libstdc++`, which is tens of thousands of lines of templates. The user's actual solution is fifty lines. Optimisation level affects the small part and not the large one.

I measured this rather than assuming it, which is why I noticed. The real fix was therefore not a flag but a **precompiled header**: build `stdc++.h.gch` once and reuse it, taking 5.0 s down to 2.8 s.

Two implementation details worth mentioning. The `.gch` is **~139 MB and only valid for the exact flags that produced it**, so `_PCH_FLAGS` is pinned to `(" -O2",)` and the cache is keyed on compiler path, standard and flags — otherwise every flag variation spawns another 139 MB artefact. And GCC only looks for `<header>.gch` beside the header it actually resolved, so the build has to locate the real `bits/stdc++.h` via the preprocessor rather than guessing a path.

Q39 Your tracker scan went from over 120 seconds to 190 ms. What was the bug?

I opened the workbook with `read_only=True` — which sounds like the safe, fast choice — and then used `ws.cell(row, col)` for random access.

The problem is that read-only worksheets in openpyxl are **row streams, not arrays**. Each `ws.cell()` call re-reads from the top of the sheet to reach the requested row. Doing that across thousands of rows $\times$ five blocks makes the scan quadratic in the row count, and the first run hung past my 120-second timeout.

The fix was a **single** `iter_rows(values_only=True)` **pass**, handling all five blocks per row as it goes. One traversal, 190 ms — a 600 $\times$ improvement, and the code is shorter.

The lesson I would draw, because it generalises past openpyxl: **“read-only” described the access mode, not the performance profile**. I assumed a flag named for safety implied an in-memory structure, and the library was doing exactly what it documented. Now the function carries a comment explaining why random access is forbidden there, so the next person does not reintroduce it.

Q40 Why polling at 0.4s rather than inotify or the watchdog package?

To keep the dependency count at one. `watchdog` would be a second third-party package, and its platform backends differ — `inotify` on Linux, `ReadDirectoryChangesW` on Windows, `FSEvents` on macOS — which means it can fail *differently* on the platform I cannot test. For a feature whose entire job is “notice a file changed”, that is a bad trade.

Polling `stat()` every 400 ms costs effectively nothing for one file and behaves identically everywhere. 400 ms is below the threshold where a save feels like it did not register.

The `DEBOUNCE_SECONDS = 0.6` is the more interesting number. Editors do not write files atomically — many write in chunks, or write to a temp file and rename — so a single save can produce several mtime changes. Without debouncing, the watcher fires mid-write and compiles a truncated file, producing a syntax error that vanishes on the next poll. Debouncing waits for the file to stop changing before acting.

I stamp both mtime *and* size, since mtime alone has coarse resolution on some filesystems and can miss a fast rewrite.

Q41 What is the worst-case cost of a single ``solve`` command? Can you state it before running?

Yes — and being able to is the point of capping the one cycle.

`solve` is: fetch (no model), driver-repair (≤ 3 attempts), solver (≤ 3), then the Verifier $\neq$ Debugger loop (≤ 3). That is at most **9 model calls**. At roughly 1,300 tokens each, about **11,700 tokens** — slightly over one minute of free-tier budget, so the worst case is a short wait, not a failure.

The structural reason I can answer at all: every loop in the system is bounded, and there is only one of them. A system with dynamic planning would have to answer “until the agent decides it is done”, which is not a number.

What I have *not* done, and would: enforce it. The cap is per-agent (`MAX_ATTEMPTS`), not per-command, so nothing measures or limits the total. A token budget carried on the Context and decremented by each provider call would make the bound real rather than arithmetic — and would let the system fail with “budget exhausted” instead of hitting a rate limit and surfacing a provider error.

Q42 Agents request a tier, not a model id. What incident made that necessary?

A model was **retired underneath me**. `llama-3.3-70b-versatile` stopped being served, and every model-backed agent started failing with an unhelpful provider error. Free-tier catalogues change without notice, and I had hardcoded a name.

Two changes came out of it. Agents now ask for `fast` or `smart`, and the mapping to concrete ids lives in `config.toml` — so recovering from a retirement is editing one line, not fifteen files. And I added `provider models`, which queries the live endpoint for what it actually serves, so the next time this happens the diagnosis takes seconds rather than guesswork.

The wider payoff is that the same indirection makes **provider switching** free. The order `groq` $\rightarrow$ `gemini` $\rightarrow$ `cerebras` $\rightarrow$ `openrouter` $\rightarrow$ `ollama` $\rightarrow$ `anthropic` works because no agent knows which one it is talking to — and the paid provider being last is what stops a misconfiguration silently spending money.

The general principle: **depend on the capability you need, not on the name of the thing currently providing it**. That is ordinary dependency inversion; it just happens to matter more when the dependency is a third party's product roadmap.

Failure & Debugging

Seven real bugs, each with a diagnosis and a lesson that generalises.

The bugs are the most interviewable material you have. Each one is a story with a diagnosis and a general lesson — which is exactly the shape of answer senior interviewers are listening for.

7.1 Seven real bugs

Bug	Why it was hard	General lesson
Empty model response	no exception, valid response, empty content	semantic failures need their own assertions
Scorer: “no reference”	staging dir and binary shared a name; the linker overwrote its own directory	resource-name collisions look like logic bugs
Retired model id	worked yesterday, failed today, nothing changed locally	depend on capabilities, not names
Quadratic tracker read	a flag named for safety implied the wrong perf model	read the access-pattern docs, not the flag name
Rollback deleted evidence	correct by the textbook, wrong for the user	“atomic” needs a definition
ok() kwarg collision	only fired on the dry-run path	flexible signatures hide arity bugs
A solved problem 0	the arithmetic was right, the data was odd	validate values, not just addressing

7.2 Questions

Q43 Tell me about the hardest bug.

A model returning **empty content** — and it is the best story I have, because every layer of conventional error handling passed it through.

Symptom: agents intermittently produced nothing. No exception, no timeout, HTTP 200, valid JSON, schema satisfied. The `content` field was an empty string, and downstream that became “the model had no suggestion” — a plausible, completely wrong conclusion.

Diagnosis: the free model is a *reasoning* model. It can spend its entire completion budget on internal reasoning tokens and emit no visible output. The request succeeded; the budget was simply exhausted before it started answering.

Fix, in three parts: set `reasoning_effort="low"`; enforce a `MIN_REASONING_BUDGET` of 1,500 tokens so there is room to finish; and — the only part that is a real safeguard — **raise `ProviderError` on empty content** rather than returning it.

Lesson: in agent systems the dangerous failures are *semantic*, not infrastructural. Retries, timeouts and circuit breakers handle failures that announce themselves. A response that is well-formed and meaningless announces nothing, so it needs an assertion written specifically for it. I now treat “valid but empty” as a first-class error case everywhere.

Q44 Your scorer reported 'no reference' when a reference existed. What was happening?

A **name collision** that presented as a logic bug, which is why it took a while.

To measure efficiency, the scorer copies the archived reference solution into a staging directory, compiles it, and times both binaries. Both the staging directory and the output binary were named `ref_<pid>`. So the linker was asked to write an executable at a path that was already a directory — it failed, the compile step returned unsuccessfully, and the scorer concluded there was no usable reference.

What made it hard: the symptom pointed at the *archive*. I checked the file existed, checked the path resolution, checked the glob. The actual failure was two lines away in a subprocess whose stderr I was discarding on that path.

Fix: distinct names — `refstage_<pid>` and `refbin_<pid>`. Trivial once seen.

Two lessons. **Surface subprocess stderr**, even on paths you expect to succeed — I was throwing away the exact message that would have solved it. And **derive names from distinct prefixes** when a scheme generates both files and directories, because the namespace is shared and collisions are silent.

Q45 Describe a bug that was your own mistaken diagnosis.

Worth having one ready, because it demonstrates you check yourself.

I was validating that the scorer discriminates between approaches. The optimal binary-search solution for problem 4 scored 94/A, which was right. Then a later run of the same file scored **73.5/C**, with complexity at 5/25 — the signature of a brute force. My first conclusion was that the scorer was non-deterministic or that the model was inconsistent, and I started looking at the scoring path.

It was neither. The file on disk had changed: an editor had written back a **stale buffer** containing the earlier merge-and-sort version. The scorer had read the file correctly and graded it correctly. The 73.5/C was a true statement about different code.

How I caught it: before changing anything, I read the file the scorer was actually reading rather than the one I believed was there. That is the habit worth naming — **verify the input before debugging the process**.

It also had a real consequence I only found later: that stale version had been archived, which poisoned the reference corpus for problem 4 (Q36).

Q46 You had a `TypeError` that only fired on the dry-run path. How did that happen?

`AgentResult` is built by a helper, `ok(self, message: str, **data)`. The committer wanted to put its proposed commit message into the structured data, so it called `self.ok("would commit", message=msg)`. Python bound that `message=` to the positional parameter rather than collecting it into `**data`, and raised `TypeError: got multiple values for argument 'message'`.

It only fired on the dry-run path because that is the only path that proposes a message without committing — the real commit path passes it differently. So the common path was fine and the rarely-exercised one crashed.

Fix: rename the data key to `commit_message`, then **audit every `self.ok(...)` call site** in the codebase for the same shape. No others.

The design lesson is the interesting part: **`**kwargs` made an arity error invisible until runtime**. A signature of `ok(self, message: str, data: dict = None)` would have made the collision impossible by construction. I kept the flexible form because it reads better at every call site, which is a real trade — but I would now say the convenience is not worth a class of error that hides on cold paths.

Q47 How do you debug this system when something goes wrong?

Honestly: **less well than I would like**, and I would rather say that than describe an observability stack I do not have.

What exists: `ctx.history` — every agent appends a timestamped line, and `status` prints the trail, so I can see which agents ran in what order. Agent results carry structured `data`, so failures report why rather than just failing. Errors name the agent. And most of the system is deterministic, so most bugs reproduce.

What is missing: **no per-call token counts, no latency, no retry counts, no record of which degradation path was taken, no persisted traces**. When the empty-response bug appeared I was reading raw provider responses by hand, because nothing was logging them.

The specific thing I would change if rebuilding: **structured tracing from phase 0**. `ctx.history` was added when debugging got painful, which is already too late — and it records prose, not fields. Per-agent records with tokens, latency, retries and the path taken would have made three of the seven bugs immediately obvious instead of investigations.

The general point: in a system with any non-determinism, **tracing is not a debugging aid, it is the debugger** — and correlation cannot be retrofitted.

Weaknesses & Scale

The three questions that decide how senior you sound.

The last three questions decide how senior you sound. Volunteering a weakness before it is found, with a concrete fix and a reason you did not do it, is the strongest signal available to you.

8.1 What you must volunteer

Weakness	Say this	Do not say
Never run on Windows	“Written for it — no bash, pathlib throughout, CRLF launchers, MinGW path hunting — but never executed there. I wrote a setup script whose --check mode is the fastest way to find out.”	“It's cross-platform.”
No eval set	“Testing was manual. A fixed set of problems with graded outcomes would have caught scorer regressions and made prompt changes measurable.”	“I tested it thoroughly.”
Efficiency scores brute force full marks	“Right for the wrong reason on one axis. Complexity compensates, and it's documented as a known gap.”	silence, then being caught
No stress-test generator	“The single highest-value thing missing. It would fix efficiency scoring and make ‘verified’ mean more than two examples.”	“The verifier proves correctness.”
Tracing added late	“ctx.history exists but has no token counts or latency. I'd build structured tracing in phase 0 next time.”	describing observability you do not have

8.2 Questions

Q48 What are the weaknesses of this system?

Five, and I would lead with the one that undermines a stated requirement.

1. It has never run on Windows, despite that being an explicit requirement. Every known trap is designed around — no bash dependency, `pathlib` throughout, `subprocess(timeout=)` rather than `signal.alarm`, CRLF launchers, MinGW path hunting — but “avoids the known traps” is not “verified”. The most likely first failure is the compiler hunt.

2. No stress-test generator. The verifier checks two example cases, so “verified” is weaker than it sounds, and the efficiency component gives an $O(n^2)$ brute force full marks because the examples are too small to differentiate. This is the highest-value thing missing.

3. No evaluation set. Testing was manual. Prompt changes were assessed anecdotally.

4. Tracing was added late and lacks tokens, latency and retry counts.

5. Single-user by construction — local files, single-writer assumptions.

I would then say which one I would fix first and why: the stress-test generator, because it is the only one that would make an existing component *correct* rather than merely better-instrumented.

Q49 Scale this from one user to a thousand. What breaks?

The architecture survives; the storage and isolation assumptions do not. I would be specific about which is which.

Breaks immediately:

- **State.** `session.json`, the cache and the `.xlsx` are single-writer local files. They become per-user rows in a database, and the tracker stops being a spreadsheet.
- **Rate limits.** A free tier serving one user does not serve a thousand. That needs paid capacity, a queue, and per-user quotas — and the cost model changes from zero to something budgeted per user, which changes the product.
- **Compilation.** This is the hard one. Running arbitrary user-supplied C++ needs a sandbox with CPU, memory and wall-clock limits, no network, and a disposable filesystem. It is the single biggest piece of work in the migration, and it is a security problem rather than a scaling one.

Survives unchanged: the agent contract, the two-tier split, the pipelines, and the generate-verify-retry loop. The blackboard actually helps — one Context per user preserves the single-writer assumption exactly, so concurrency between users needs no new coordination.

What I would *not* do is decompose into microservices per agent. The agents are milliseconds apart in one process; the network boundary would cost more than it buys.

Q50 If you rebuilt it today, what would you do differently?

Three changes, in the order I would make them.

1. A stress-test generator, in phase 0. Random and worst-case input generation checked against a brute-force oracle. It would fix the efficiency component, make “verified” mean something stronger than two examples, and strengthen the improver’s claim. I proved the value accidentally — stress-testing one solution over 214,348 random cases found zero mismatches and gave me confidence nothing else in the system could.

2. Structured tracing from the first commit. Not `ctx.history`’s prose lines, but per-agent records: tokens, latency, retries, degradation path, model. Three of my seven bugs would have been immediately visible instead of investigations, and correlation cannot be retrofitted.

3. An evaluation set before the first model-backed agent. A fixed set of problems with known-good outcomes, graded automatically. Without it, every prompt change was assessed by running it once and looking — which is not evidence.

What I would **keep**, and I would say this explicitly: the blackboard, the two-tier split, and the deterministic verifier gating every generator. Those three held up under everything, including requirements that arrived after the design was set. The pattern is that all three are *constraints* rather than features — and the constraints are what made the rest tractable.

THE CLOSING LINE

If you get one sentence to summarise what you learned, use this: *“The thing that made this work wasn’t making the agents smarter — it was making their failures cheap. A deterministic verifier, a bounded retry, a backup before every write, and a declared capability so the system shrinks instead of breaking.”*

Phase 7 — the archive first, and the answer withheld

Eight new questions · 12 September 2026 · every number measured on the 3,356-problem archive

The book describes the system as it stood on 11 September. One change since then touches Part 1's numbers, Part 4's verification story, and the answer to Q18 directly. This addendum records what changed, what it was measured against, and the questions the change invites. Nothing in the original pages is withdrawn except where §A.3 says so.

A.1 What changed

	Before	After
Menu option 1	LeetCode first; the local archive only when the fetch failed	the local archive first; LeetCode only for what the archive does not have
The solution file	the archive's <code><id>.cpp</code> — the finished solution, handed back	an empty stub rebuilt from it: signatures kept, bodies emptied
The question file	rendered from the API — a network call unless cached	<code><id>_problem.txt</code> sits in all 3,356 archive folders
The driver	two shapes: plain call, or ops/args replay	matched to the code type — see §A.2

The re-solve loop is the point. An archived problem now scaffolds with no network at all: the driver, the tests and the question come off disk, and the one file that would spoil the exercise — the solution — is the one file that is rebuilt empty.

A.2 The numbers

Measurement	Result
Question files written into <code>Code Dirs/</code>	3,356 of 3,356, 0 failures — one API request each, the slug taken from the solution's <code>// Link:</code> line
Stubs compiled against their own	3,355 of 3,355 compile. The one folder without a stub, 3795,

archived driver	holds four empty files, so the fetcher falls through to LeetCode
Code shapes across the archive	function 3,195 · design 134 · interactive 20 · concurrency 6
Structures crossing a signature	tree 148 · linked list 47 · n-ary 9 · nested list 4 · next pointers 3 · parent pointers 3 · random list 2 · quad tree 2 · multilevel 1 · graph 1
Generator differential test (3,083 problems with a cached snippet)	3,036 byte-identical · 40 fixed · 0 regressions · 7 still failing

Fig A.1 — every figure in this addendum is a measurement, not an estimate. The method for the last row is Q56.

A.3 A correction to Q18

Q18 answers the charge that the fetcher needs the network by saying it “tries the on-disk cache first, then the local archive of ~3,350 solutions, and only then the network”. That described the degradation I had designed. The code did not do it: it called LeetCode first and reached the archive only when that call failed, which is the opposite order and means the good data was reached by accident rather than by design.

The code now does what the answer says. If this comes up, the version worth giving is the unflattering one: I described the behaviour I intended, found the implementation inverted when I went to change something else, and fixed it. The answer's conclusion — that an archived problem works with no connection — is true now, and is stronger than it was: the question file travels with the scaffold, so there is no network call left in the path at all.

A.4 Questions

Q51 Your fetcher went to the network first and treated your own archive as a fallback. Why was that the wrong way round?

Three costs, in order of weight. **The archive holds the repaired drivers.** Those 3,356 folders are not a cache of what LeetCode would give me — they are what I had to fix: hand-written readers for odd input notations, self-validating harnesses for special judges, corrected expected files. Fetching fresh regenerates the broken version of exactly the problems that cost the most work.

It put a network call in the common case. The common case is re-solving a problem I have already done, which is what the archive is for.

A fallback that only runs on failure is untested. It ran when LeetCode was down, which is rarely, so its behaviour was the least exercised path in the system.

What I gave up: the archive can hold a stale scaffold — 430's input file is literally the word placeholder — so `--live` skips the archive and fetches fresh, and a problem whose archive entry is

missing or empty falls through to LeetCode automatically, saying which files were missing.

Q52 You hand back an empty stub instead of the stored solution. How do you know the stub is correct?

Because correctness here has a mechanical definition: the stub must compile against the same driver the archive ships beside it. So I compiled all of them. Every stub the archive can produce — 3,355 of them — compiles against the driver archived beside it.

The first run passed 3,351 of 3,356, and the five failures were the useful part. Two signatures used `ll`, an alias declared inside the class body I was deleting. One carried a user-added default argument (`int ans = INF3455`) that is not part of LeetCode's API. One kept an out-of-class `vector<int> Solution::v`; definition for a member that no longer existed. One had a template default argument (`uint8_t RBITS = SBITS`) that my segmenter read as an initialiser, so it swallowed the class that followed. The fifth was not a bug: 3795's archive entry is four empty files, and there the fetcher correctly falls through to LeetCode.

What the test does not prove is that the stub is the signature LeetCode would give — only that it is the signature the driver calls. Those agree for every problem the archive has, because the driver was generated from LeetCode's own snippet.

Q53 Why re-derive the stub every time instead of storing one per problem?

A stored stub is 3,356 more files that can drift from the driver sitting next to them. The derivation reads the driver to decide which methods are the API — whatever it calls survives, private helpers and members do not — so the stub cannot disagree with the harness that calls it.

It also buys a property I use elsewhere. The rebuild is idempotent, so `make_stub(x) == x` is exactly the test for “this file is still the untouched stub”. That is how clearing the root knows not to rescue an empty file into `data/replaced/`, where it would overwrite a genuine rescued solution with the same id. No marker file, no extra state, no way for the two to disagree.

Q54 Why is the judge's mock in the driver and never in the file you submit?

284 taught me this one, by failing on the judge rather than locally. Its `Iterator` base class is supplied by LeetCode at submission time. A concrete `class Iterator {...}` inside `284.cpp` compiles perfectly well on my machine and is a **redefinition error** the moment the file is submitted.

So the mock goes in the driver, above its `#include "<id>.cpp"`: only the local harness ever sees it, and the file I submit declares nothing the judge already provides. `NestedInteger` is handled the same way. Hidden APIs are the mirror image — the stub carries the declaration `int guess(int num);`, which a judge accepts as a redeclaration, and the driver carries the definition backed by that case's extra input line.

The cost is IDE diagnostics on the solution file alone, which cannot see the base class. That is noise I

accept; the alternative is noise the judge rejects.

Q55 LeetCode calls six different structures Node. How do you tell them apart, and why does it matter?

By their fields, most specific first: `neighbors` is a graph vertex, `children` an n-ary tree, `random` a list with random pointers, `child` a multilevel list, `topLeft` a quad tree, and a bare `next` beside `left/right` a tree with next pointers.

It matters because the name carries no information and each one arrives in a different notation: an adjacency list, a level order with nulls closing each child group, `[[val, random_index]]`, levels separated by null runs, `[isLeaf, val]` pairs. Reading one as another does not crash — it silently builds the wrong structure. Every variant exposes the same three names (`_toNode`, `_sNode`, `_rnode`), so the type tables need one entry for `Node*` and the variant is chosen once, from the definition.

The quad tree needed one extra piece of honesty. LeetCode states that a non-leaf node's `val` is not compared, and its own examples disagree with each other — 427 prints a 1 there, 558 prints a 0. The printer emits `[0, *]` for a non-leaf and the expected file is normalised the same way, so the test checks the structure and not a value the judge ignores.

Q56 You changed a generator that 3,356 problems depend on. How did you know you had not broken it?

Differential testing against the previous version, not judgement. I kept the old generator, ran both over every problem with cached metadata — 3,083 of them — and compared the files they wrote. For 3,036 the output is byte-identical apart from helper functions the new one adds, so the behaviour cannot have changed and no compilation is needed. That shortcut is what makes the test affordable.

For the 47 that differ I compiled and ran both: each generated driver was fed the archived *solved* solution and its output diffed against LeetCode's own expected values. 40 pass that did not before — the six `Node` variants, `NestedInteger`, `guess`, `isBadVersion`, the `Codec` round trips, `PeekingIterator` — and nothing regressed.

The test also found a gap I had not gone looking for: 19 problems return `vector<bool>`, which the generator had never been able to print.

Q57 What still does not work?

Seven problems fail on the LeetCode path, and all seven failed before. Three (384, 519, 470) return random values, so no fixed expected file can match them; they need a self-validating driver, which is a per-problem judgement and not something a generator should guess. Three (843, 1095, 1237) use judge classes I do not mock — `Master`, `MountainArray`, `CustomFunction` — and there the fetcher prints a warning naming the class and where the mock belongs. One (1462) is an output mismatch I have not diagnosed.

Concurrency problems get a driver that refuses to pretend: it compiles, prints that a thread harness is needed, and exits non-zero. A generated harness would have to know what the input means for each problem — thread start order for 1114, a count for 1115, a water string for 1117.

All ten are in the archive with working drivers, so option 1 — the path actually used — handles every one of them.

Q58 You wrote something that reads C++ to do this. Is that not the wrong tool for the job?

It is not a parser and I would not defend it as one. It blanks comments, string and character literals and preprocessor lines, then scans braces and parentheses — enough for the shapes LeetCode solutions take, and nothing beyond them.

Two things make that acceptable. The **failure mode**: when it cannot find a solution class it returns nothing, and the fetcher falls through to LeetCode rather than writing a file that is subtly wrong — the same rule as the rest of the fetcher, which modifies nothing when it fails. And the **test set**: 3,356 real solutions, every one compiled, including the deliberately pathological ones — a copied solution carrying AVX intrinsics and attribute macros, classes using tabs, a solution whose helper is defined outside the class.

The alternative is libclang: correct by construction, and a dependency that has to exist on Windows too, which is a requirement this project actually has. The measured benefit would have been five files out of 3,356, all five fixable in an afternoon. I would revisit that decision the first time a sixth class of failure appears rather than defending the scan on principle.